

Web Server HTTP/1.0 in C

Emanuele De Francesco

Giugno 2026

Obiettivo del progetto

Realizzazione di un Web Server HTTP/1.0 in linguaggio C in grado di:

- Accettare connessioni TCP
- Interpretare richieste HTTP
- Servire file statici
- Gestire più client contemporaneamente
- Registrare i log in modo asincrono

Sviluppo incrementale del progetto

Il server è stato sviluppato in più fasi:

- 1 Comunicazione TCP
- 2 Parsing HTTP
- 3 File Serving
- 4 Gestione concorrente con `select()`
- 5 Logging asincrono

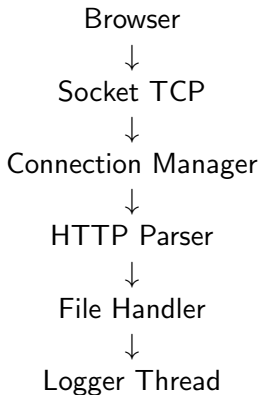
Ogni fase è stata verificata singolarmente prima di passare alla successiva.

Evoluzione dell'architettura

Versione iniziale:

Browser → Socket TCP → Risposta statica

Versione finale:



Struttura del progetto

- `server.c`
- `create_socket.c`
- `connection_manager.c`
- `client_handler.c`
- `http_parser.c`
- `file_handler.c`
- `logger.c`

Codice suddiviso in moduli indipendenti per migliorare leggibilità e manutenzione.

Responsabilità principali:

- Avvio del server
- Creazione della socket TCP
- Avvio del thread logger
- Avvio del Connection Manager

Punto di ingresso dell'applicazione.

Gestione della creazione della socket TCP.

Funzioni utilizzate:

- socket()
- bind()
- listen()

Restituisce il file descriptor del server pronto all'ascolto sulla porta 8080.

Gestione delle connessioni concorrenti.

Funzionalità principali:

- Utilizzo di select()
- Monitoraggio delle socket
- Gestione dei nuovi client
- Invocazione del Client Handler

Permette la gestione simultanea di più connessioni.

Gestione delle richieste HTTP.

Operazioni principali:

- Ricezione dati tramite `recv()`
- Chiamata al parser HTTP
- Chiamata al File Handler

Collega la parte di rete alla logica applicativa.

Analisi della Request Line HTTP.

Esempio

```
GET /index.html HTTP/1.1
```

Informazioni estratte:

- Metodo
- Path
- Versione HTTP

Implementazione tramite `sscanf()`.

Responsabile del File Serving.

Funzionalità:

- Apertura file tramite `fopen()`
- Gestione 200 OK
- Gestione 404 Not Found
- Gestione 405 Method Not Allowed
- Gestione Content-Type

Invio dei file richiesti al browser.

Sistema di logging asincrono.

Componenti:

- Thread dedicato
- Coda circolare
- Mutex
- File access.log

La scrittura dei log avviene separatamente dal thread principale.

Gestione concorrente con select()

Problema:

- Un server iterativo gestisce un solo client alla volta

Soluzione:

- Utilizzo della funzione select()
- Monitoraggio di più socket contemporaneamente
- Nessun thread per ogni client

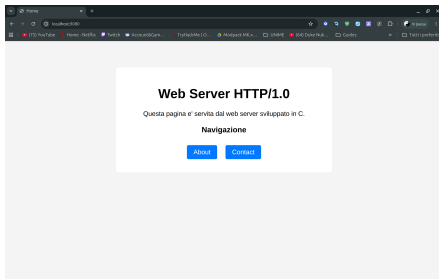
Componenti principali:

- Thread dedicato
- Coda condivisa
- Mutex di protezione
- File access.log

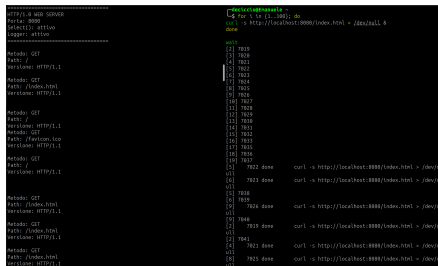
Il server continua a rispondere ai client mentre il logger salva i messaggi su disco.

Test effettuati

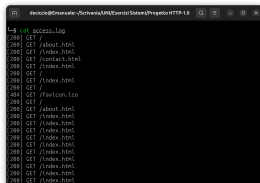
Browser Test



Test Concorrente



Verifica del Logger



Funzionalità implementate:

- TCP Socket
- Parsing HTTP
- File Serving
- Gestione concorrente tramite `select()`
- Logging asincrono con thread e mutex

Possibili sviluppi futuri:

- Supporto POST
- Utilizzo di `epoll()`
- Supporto HTTPS
- Gestione contenuti dinamici

Grazie per l'attenzione